## (a) Translator Prompt

```
You are given code that solves an algorithmic
problem. Reason through the problem step-by-step
using natural language and arrive at the answer.
Do NOT translate the code mechanically.
```

**GUIDELINES**

- Think like a human (exploratory reasoning)
- Be conversational ("Let me check...", "I notice")
- Skip obvious steps, focus on insights (WHY)
- Use natural structure (paragraphs over lists)

**10 IN-CONTEXT EXAMPLES**

```
Example: Topological Sort
  Input:  Adjacency matrix A = [[0,1,0,...],...]
  Output: "Node 3 has in-degree 0... Answer is 3."
(+ 9 more: KMP, Bridges, LCS, Bellman-Ford, ...)
```

**TEST INPUT**

```
def solution():  # [Code to translate]
```

## (b) Discriminator Prompt

```
You are analyzing an explanation of how to
solve an algorithmic problem.

TASK: Determine whether this was written by
someone solving naturally ("Native NL") or
translating/simulating code ("Translated").
```

**INPUT FORMAT**

```
PROBLEM:
  {Algorithmic problem description}
EXPLANATION:
  {Reasoning trace to classify}
```

**OUTPUT FORMAT**

```
PREDICTION: [NATIVE or TRANSLATED]
CONFIDENCE: [HIGH, MEDIUM, or LOW]
REASONING:  [1-2 sentence justification]
```